

Projeto Programação Concorrente e Paralela

Carlos Eduardo Nogueira Silva
Felipe Gomes da Silva
Gabriel Martins Brum
Luis Henrique Salomão Lobato

Setembro, 2025

1 Introdução

Sistemas de equações lineares ($Ax=b$) podem ser resolvidos de diferentes formas, sendo que um dos possíveis métodos de convergência para identificar soluções é o método de Jacobi, uma abordagem de iterações.

Neste trabalho foram medidos os tempos de execução a partir da implementação do método de Jacobi tanto sequencial, quanto em paralelo através do uso da ferramenta openMP.

2 Metodologia

2.1 Cálculo

O método iterativo de Jacobi foi representado algebricamente através da composição de uma matriz diagonal e as matrizes superiores e inferiores da matriz A. Através desta definição, podê-se chegar em uma solução para um sistema linear através da aplicação da equação 1.

O critério de parada adotado foi o de que diferença máxima entre X_i^{k+1} e X_i^k fosse de 10^{-5} .

$$x_i^{k+1} = \frac{1}{a_{ii}} \left(b_i - \sum_{j \neq i} a_{ij} x_j^k \right) \quad (1)$$

2.2 Testes

Foram testadas tanto a implementação sequencial do método, quanto como a execução paralela do algoritmo, com diferentes parâmetros de ajuste.

O contexto de aplicação dos testes respeitou matrizes A com dimensões de 2000 x 2000 e por consequência, vetores b com 2000 constantes, de modo que os valores utilizados foram de números reais de até 4 casas decimais, respeitando o intervalo de -500 a 500.

Os parâmetros avaliados ao longo dos testes com a versão openMP foram o número de threads de execução variando de 2 a 8, o tipo de escalonador de threads (*static*, *guided* e *dynamic*) e o número de *chunks* (0 ou 25).

Os testes foram executados em diferentes máquinas com diferentes processadores, visando evitar qualquer tipo de viés na análise de desempenho dos métodos e aumentar o contexto de exploração proposto.

3 Resultados

Na Tabela 1 estão detalhados os processadores utilizados para a executar dos programas sequencial e openMP, seguido pelo número de núcleos e *threads* dos mesmos.

Tabela 1: Especificações dos processadores onde os algoritmos foram testados.

Processador	Núcleos	Threads
Intel Core i3-10100F	4	8
Intel Core i7-10750H	6	12
AMD Ryzen 7 5700u	8	16
AMD Ryzen 5 3500u	4	8

Assim, encontram-se nas Tabelas 2, 4, 3 e 5 os resultados das medições do tempo de execução dos algoritmos em cada processador.

Tabela 2: Resultados da execução dos testes em um i3-10100F.

Threads	Schedule	Chunk	Tempo de Execução
1 (sequencial)	N/A	N/A	0.1024s
2	Estático	N/A	1.1503s
4	Estático	N/A	2.6165s
8	Estático	N/A	3.5367s
2	Estático	25	1.1271s
4	Estático	25	2.3659s
8	Estático	25	2.6648s
2	Dinâmico	25	1.1453s
4	Dinâmico	25	2.1047s
8	Dinâmico	25	2.8254s
2	Guided	25	1.0959s
4	Guided	25	2.6802s
8	Guided	25	3.1195s

Tabela 3: Resultados da execução dos testes em um i7-10750H.

Threads	Schedule	Chunk	Tempo de Execução
1 (sequencial)	N/A	N/A	0.4097s
2	static	1	4.3661s
4	static	1	8.7037s
8	static	1	11.7228s
2	static	25	2.9943s
4	static	25	7.0912s
8	static	25	10.9250s
2	dynamic	25	4.0930s
4	dynamic	25	8.7469s
8	dynamic	25	11.4043s
2	guided	25	4.0025s
4	guided	25	9.1439s
8	guided	25	13.2075s

Tabela 4: Resultados da execução dos testes em um AMD Ryzen 7 5700u.

Threads	Schedule	Chunk	Tempo de Execução
1 (sequencial)	N/A	N/A	0.1277s
2	Estático	1	1.0148s
4	Estático	1	2.0533s
8	Estático	1	2.6728s
2	Estático	25	1.0125s
4	Estático	25	2.2540s
8	Estático	25	2.6751s
2	Dinâmico	25	1.0092s
4	Dinâmico	25	2.0855s
8	Dinâmico	25	2.6796s
2	Guided	25	1.0120s
4	Guided	25	2.1032s
8	Guided	25	2.6918s

Tabela 5: Resultados da execução dos testes em um AMD Ryzen 5 3500u.

Threads	Schedule	Chunk	Tempo de Execução
1 (sequencial)	N/A	N/A	0.1515s
2	static	1	1.0952s
4	static	1	2.7601s
8	static	1	4.3522s
2	static	25	1.0758s
4	static	25	2.7876s
8	static	25	4.4914s
2	dynamic	25	1.0862s
4	dynamic	25	2.8168s
8	dynamic	25	4.2930s
2	guided	25	1.4712s
4	guided	25	2.5914s
8	guided	25	3.8268s

4 Conclusão

A partir dos dados coletados, foi possível concluir que para este caso específico de testes, onde o método de Jacobi acabou por convergir em poucas iterações (neste caso, 7), a implementação sequencial obteve vantagem de forma majoritária, pois o programa openMP acaba por consumir mais tempo para a criação, alocação e escalonamento de threads do que executando em si o cálculo do método.